

Handling HTTP Requests With Ease

# React Query / Tanstack Query

## Easy Guide

# Contents

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| <b>1</b> | <b>React Query: What &amp; Why?</b>                           | <b>2</b>  |
| 1.1      | What is React Query? . . . . .                                | 2         |
| 1.2      | The Problem Without React Query . . . . .                     | 2         |
| 1.3      | With React Query . . . . .                                    | 2         |
| 1.4      | Why Use React Query? . . . . .                                | 3         |
| 1.5      | Key Concepts . . . . .                                        | 3         |
| 1.5.1    | Query . . . . .                                               | 3         |
| 1.5.2    | Mutation . . . . .                                            | 3         |
| 1.5.3    | Cache . . . . .                                               | 3         |
| 1.5.4    | Stale . . . . .                                               | 3         |
| <b>2</b> | <b>Installing &amp; Using Tanstack Query</b>                  | <b>4</b>  |
| 2.1      | Installation . . . . .                                        | 4         |
| 2.2      | Setup . . . . .                                               | 4         |
| 2.3      | Using Queries . . . . .                                       | 4         |
| 2.4      | What You Get Back . . . . .                                   | 5         |
| 2.5      | Why This Is Great . . . . .                                   | 5         |
| <b>3</b> | <b>Understanding &amp; Configuring Query Behaviors</b>        | <b>6</b>  |
| 3.1      | Caching . . . . .                                             | 6         |
| 3.2      | Stale Time . . . . .                                          | 6         |
| 3.3      | Cache Time . . . . .                                          | 6         |
| 3.4      | Refetch Strategies . . . . .                                  | 7         |
| 3.5      | Default Behavior . . . . .                                    | 7         |
| 3.6      | Configuration Example . . . . .                               | 7         |
| <b>4</b> | <b>Dynamic Query Functions &amp; Query Keys</b>               | <b>8</b>  |
| 4.1      | Query Keys . . . . .                                          | 8         |
| 4.2      | Dynamic Query Keys . . . . .                                  | 8         |
| 4.3      | Query Keys With Multiple Parameters . . . . .                 | 9         |
| 4.4      | Best Practices for Query Keys . . . . .                       | 9         |
| 4.5      | Query Key Factories . . . . .                                 | 9         |
| <b>5</b> | <b>The Query Configuration Object &amp; Aborting Requests</b> | <b>10</b> |
| 5.1      | Query Configuration . . . . .                                 | 10        |
| 5.2      | Aborting Requests . . . . .                                   | 10        |
| 5.3      | Manual Abort . . . . .                                        | 11        |
| 5.4      | Retry Configuration . . . . .                                 | 11        |

|           |                                                                 |           |
|-----------|-----------------------------------------------------------------|-----------|
| <b>6</b>  | <b>Enabled &amp; Disabled Queries</b>                           | <b>12</b> |
| 6.1       | Enabling/Disabling Queries . . . . .                            | 12        |
| 6.2       | Dependent Queries . . . . .                                     | 12        |
| 6.3       | Common Use Cases . . . . .                                      | 13        |
| 6.3.1     | Search Query . . . . .                                          | 13        |
| 6.3.2     | Authenticated Requests . . . . .                                | 13        |
| <b>7</b>  | <b>Changing Data with Mutations</b>                             | <b>14</b> |
| 7.1       | What is a Mutation? . . . . .                                   | 14        |
| 7.2       | Using Mutations . . . . .                                       | 14        |
| 7.3       | Mutation States . . . . .                                       | 15        |
| 7.4       | Mutation Data . . . . .                                         | 15        |
| 7.5       | Mutation with Variables . . . . .                               | 15        |
| <b>8</b>  | <b>Fetching More Data &amp; Testing the Mutation</b>            | <b>16</b> |
| 8.1       | Fetching in Stages . . . . .                                    | 16        |
| 8.2       | Testing Mutations . . . . .                                     | 16        |
| 8.3       | Manual Fetch . . . . .                                          | 17        |
| <b>9</b>  | <b>Acting on Mutation Success &amp; Invalidating Queries</b>    | <b>19</b> |
| 9.1       | Callbacks on Success . . . . .                                  | 19        |
| 9.2       | Invalidating Queries . . . . .                                  | 19        |
| 9.3       | Complete Example . . . . .                                      | 20        |
| 9.4       | Partial Invalidation . . . . .                                  | 20        |
| <b>10</b> | <b>Disabling Automatic Refetching After Invalidations</b>       | <b>22</b> |
| 10.1      | Default Behavior . . . . .                                      | 22        |
| 10.2      | Disabling Refetch . . . . .                                     | 22        |
| 10.3      | Refetch Types . . . . .                                         | 22        |
| 10.4      | Example: Optimistic Approach . . . . .                          | 23        |
| 10.5      | When to Use . . . . .                                           | 23        |
| <b>11</b> | <b>Enhancing the Demo App &amp; Repeating Mutation Concepts</b> | <b>24</b> |
| 11.1      | Complete Todo App Example . . . . .                             | 24        |
| 11.2      | Key Mutation Concepts . . . . .                                 | 26        |
| <b>12</b> | <b>React Query Advantages In Action</b>                         | <b>27</b> |
| 12.1      | Before: Manual Fetching . . . . .                               | 27        |
| 12.2      | After: React Query . . . . .                                    | 27        |
| 12.3      | Advantage 1: No State Management . . . . .                      | 28        |
| 12.4      | Advantage 2: Automatic Caching . . . . .                        | 28        |
| 12.5      | Advantage 3: Automatic Refetching . . . . .                     | 28        |
| 12.6      | Advantage 4: Easy Mutations . . . . .                           | 28        |
| 12.7      | Advantage 5: Better DevTools . . . . .                          | 28        |
| <b>13</b> | <b>Updating Data with Mutations</b>                             | <b>29</b> |
| 13.1      | Update Pattern . . . . .                                        | 29        |
| 13.2      | Delete Pattern . . . . .                                        | 29        |
| 13.3      | Bulk Update . . . . .                                           | 30        |

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| <b>14 Optimistic Updating &amp; Query Keys with React Router</b> | <b>31</b> |
| 14.1 Optimistic Updating . . . . .                               | 31        |
| 14.2 Using Query Keys with React Router . . . . .                | 32        |
| 14.3 Route Change Refetch . . . . .                              | 32        |
| 14.4 Complete Example . . . . .                                  | 32        |

# Chapter 1

## React Query: What & Why?

### 1.1 What is React Query?

React Query (now called TanStack Query) is a library that makes fetching and managing data from servers much easier.

### 1.2 The Problem Without React Query

```
const [data, setData] = useState(null);
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);
```

```
useEffect(() => {
  fetchData()
    .then(result => {
      setData(result);
      setLoading(false);
    })
    .catch(err => {
      setError(err);
      setLoading(false);
    });
}, []);
```

```
// Now manage: caching, refetching, updates,
// error handling, retry logic... lots of work!
```

### 1.3 With React Query

```
const { data, isLoading, error } = useQuery({
  queryKey: ['todos'],
  queryFn: () => fetch('/api/todos').then(r => r.json())
});
```

```
// React Query handles: caching, refetching, updates,
```

```
// error handling, retry logic automatically!
```

## 1.4 Why Use React Query?

1. **Automatic Caching** - Store fetched data automatically
2. **Background Refetching** - Keep data fresh
3. **Smart Retry Logic** - Retry failed requests
4. **Easier Mutations** - Update server data easily
5. **Less Boilerplate** - Write less code
6. **Better UX** - Loading, error, success states automatic
7. **Stale Data Handling** - Know when data is old
8. **Offline Support** - Works offline too

## 1.5 Key Concepts

### 1.5.1 Query

Getting data from server. Read-only.

### 1.5.2 Mutation

Changing data on server. Write operations.

### 1.5.3 Cache

Storing fetched data in memory for quick access.

### 1.5.4 Stale

Data that's old and should be refreshed.

**Bottom Line:** React Query = Less code + Better performance + Better UX!

## Chapter 2

# Installing & Using Tanstack Query

### 2.1 Installation

```
npm install @tanstack/react-query
```

### 2.2 Setup

In your main component (usually App.jsx):

```
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';

const queryClient = new QueryClient();

function App() {
  return (
    <QueryClientProvider client={queryClient}>
      {/* Your app here */}
    </QueryClientProvider>
  );
}
```

This wraps your entire app and enables React Query.

### 2.3 Using Queries

Fetch data with `useQuery`:

```
import { useQuery } from '@tanstack/react-query';

function Todos() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['todos'],
    queryFn: async () => {
      const response = await fetch('/api/todos');
      return response.json();
    }
  });
}
```

```

    }
  });

  if (isLoading) return <p>Loading...</p>;
  if (error) return <p>Error: {error.message}</p>;

  return (
    <ul>
      {data.map(todo => (
        <li key={todo.id}>{todo.title}</li>
      ))}
    </ul>
  );
}

```

## 2.4 What You Get Back

`useQuery` returns:

| Property                | What It Is              |
|-------------------------|-------------------------|
| <code>data</code>       | The fetched data        |
| <code>isLoading</code>  | Is data loading?        |
| <code>error</code>      | Any error that occurred |
| <code>isFetching</code> | Currently fetching?     |
| <code>isSuccess</code>  | Did it succeed?         |

## 2.5 Why This Is Great

- Automatic caching (same query = same data)
- Automatic refetching in background
- Automatic retry on failure
- Built-in loading/error states
- No manual state management

**Result:** Way less code, better experience!

## Chapter 3

# Understanding & Configuring Query Behaviors

### 3.1 Caching

React Query remembers fetched data. Visit same page again? Instant data!

```
// First visit: fetches from server (1 second)
// Second visit: instant from cache (0 seconds)
// Visit after 5 minutes: fetches again (data was stale)
```

### 3.2 Stale Time

How long data is considered fresh:

```
useQuery({
  queryKey: ['todos'],
  queryFn: fetchTodos,
  staleTime: 1000 * 60 * 5 // 5 minutes
});

// Data is fresh for 5 minutes
// After 5 minutes, data is "stale"
// Next fetch will re-request from server
```

### 3.3 Cache Time

How long to keep cached data in memory:

```
useQuery({
  queryKey: ['todos'],
  queryFn: fetchTodos,
  gcTime: 1000 * 60 * 10 // 10 minutes (formerly cacheTime)
});
```

```
// Keep data in memory for 10 minutes
// After 10 minutes, forget about it
```

### 3.4 Refetch Strategies

When to automatically refetch data:

```
useQuery({
  queryKey: ['todos'],
  queryFn: fetchTodos,
  refetchOnMount: true,           // Refetch when component mounts
  refetchOnWindowFocus: true,    // Refetch when user returns to tab
  refetchOnReconnect: true,      // Refetch when internet returns
});
```

### 3.5 Default Behavior

```
staleTime: 0           // Data always stale
gcTime: 5 minutes      // Keep in memory 5 min
refetchOnMount: true   // Refetch on mount
refetchOnWindowFocus: true // Refetch on focus
```

### 3.6 Configuration Example

```
// For frequently changing data
useQuery({
  queryKey: ['live-scores'],
  queryFn: fetchScores,
  staleTime: 0,           // Always refetch
  refetchInterval: 5000   // Refetch every 5 seconds
});
```

```
// For static data
useQuery({
  queryKey: ['settings'],
  queryFn: fetchSettings,
  staleTime: Infinity,    // Never consider stale
  gcTime: Infinity        // Never forget
});
```

**Strategy:** Configure based on how fresh data needs to be!

## Chapter 4

# Dynamic Query Functions & Query Keys

### 4.1 Query Keys

Unique identifiers for queries. React Query uses them for caching and refetching.

```
// Same key = same cache
useQuery({
  queryKey: ['todos'],
  queryFn: fetchTodos
});

// Different key = different cache
useQuery({
  queryKey: ['users'],
  queryFn: fetchUsers
});
```

### 4.2 Dynamic Query Keys

Include variables in query key:

```
function TodoDetail({ todoId }) {
  const { data } = useQuery({
    queryKey: ['todos', todoId], // Dynamic key
    queryFn: () => fetchTodo(todoId)
  });

  return <div>{data?.title}</div>;
}

// Different todoId = different query
// todoId: 1 → ['todos', 1]
// todoId: 2 → ['todos', 2]
```

## 4.3 Query Keys With Multiple Parameters

```
function Posts({ userId, status }) {
  const { data } = useQuery({
    queryKey: ['posts', userId, status],
    queryFn: () => fetchPosts(userId, status)
  });

  return <div>Posts: {data?.length}</div>
}

// userId: 1, status: 'active' → ['posts', 1, 'active']
// userId: 2, status: 'active' → ['posts', 2, 'active']
// userId: 1, status: 'archived' → ['posts', 1, 'archived']
```

## 4.4 Best Practices for Query Keys

- Start with domain: ['todos']
- Add ID: ['todos', todoId]
- Add filters: ['posts', userId, filter]
- Keep consistent: ['items', id, 'details']

## 4.5 Query Key Factories

Organize query keys:

```
const todoKeys = {
  all: ['todos'],
  lists: () => [...todoKeys.all, 'list'],
  list: (filters) => [...todoKeys.lists(), filters],
  details: () => [...todoKeys.all, 'detail'],
  detail: (id) => [...todoKeys.details(), id],
};

// Use
useQuery({
  queryKey: todoKeys.detail(todoId),
  queryFn: () => fetchTodo(todoId)
});
```

**Pattern:** Query key = cache key = refetch key!

## Chapter 5

# The Query Configuration Object & Aborting Requests

### 5.1 Query Configuration

Full options for `useQuery`:

```
useQuery({
  // Required
  queryKey: ['todos'],
  queryFn: fetchTodos,

  // Optional - Caching
  staleTime: 1000 * 60,
  gcTime: 1000 * 60 * 10,

  // Optional - Refetching
  refetchOnMount: true,
  refetchOnWindowFocus: true,
  refetchInterval: 0,

  // Optional - Behavior
  enabled: true,
  retry: 3,
  throwOnError: false,

  // Optional - Callbacks
  onSuccess: (data) => { },
  onError: (error) => { },
});
```

### 5.2 Aborting Requests

Cancel fetch requests when component unmounts:

```
useQuery({
```

```

    queryKey: ['todos'],
    queryFn: async ({ signal }) => {
      const response = await fetch('/api/todos', { signal });
      return response.json();
    }
  });

// React Query automatically handles abort signals
// If component unmounts, request is cancelled
// No memory leaks!

```

## 5.3 Manual Abort

Cancel specific request:

```

import { useQueryClient } from '@tanstack/react-query';

function MyComponent() {
  const queryClient = useQueryClient();

  const handleCancel = () => {
    queryClient.cancelQueries({ queryKey: ['todos'] });
  };

  return <button onClick={handleCancel}>Cancel</button>;
}

```

## 5.4 Retry Configuration

Automatically retry failed requests:

```

useQuery({
  queryKey: ['todos'],
  queryFn: fetchTodos,
  retry: 3, // Retry 3 times
  retryDelay: (attemptIndex) => {
    // Wait 1000ms * attempt number
    return Math.min(1000 * 2 ** attemptIndex, 30000);
  }
});

// Fails → Wait 1s → Retry
// Fails → Wait 2s → Retry
// Fails → Wait 4s → Retry
// Fails → Give up

```

**Feature:** Abort signals prevent memory leaks!

## Chapter 6

# Enabled & Disabled Queries

### 6.1 Enabling/Disabling Queries

Conditionally run queries:

```
function UserProfile({ userId }) {
  const { data } = useQuery({
    queryKey: ['user', userId],
    queryFn: () => fetchUser(userId),
    enabled: !!userId // Only run if userId exists
  });

  return userId ? <div>{data?.name}</div> : <p>Select user</p>;
}

// If userId is null/undefined: query disabled
// If userId exists: query enabled and runs
```

### 6.2 Dependent Queries

Fetch data in sequence:

```
function UserWithPosts({ userId }) {
  // First query: fetch user
  const userQuery = useQuery({
    queryKey: ['user', userId],
    queryFn: () => fetchUser(userId),
    enabled: !!userId
  });

  // Second query: fetch posts (only after user loads)
  const postsQuery = useQuery({
    queryKey: ['posts', userQuery.data?.id],
    queryFn: () => fetchPosts(userQuery.data.id),
    enabled: !!userQuery.data?.id // Wait for user to load
  });
}
```

```

if (userQuery.isLoading) return <p>Loading user...</p>;
if (postsQuery.isLoading) return <p>Loading posts...</p>;

return (
  <div>
    <h1>{userQuery.data?.name}</h1>
    <p>Posts: {postsQuery.data?.length}</p>
  </div>
);
}

```

## 6.3 Common Use Cases

### 6.3.1 Search Query

```

function Search({ searchTerm }) {
  const { data } = useQuery({
    queryKey: ['search', searchTerm],
    queryFn: () => searchAPI(searchTerm),
    enabled: searchTerm.length > 2 // Only search if 3+ chars
  });

  return <div>Results: {data?.length || 0}</div>;
}

```

### 6.3.2 Authenticated Requests

```

function SecureData({ token }) {
  const { data } = useQuery({
    queryKey: ['secure-data'],
    queryFn: () => fetchSecureData(token),
    enabled: !!token // Only fetch if logged in
  });

  return token ? <div>{data}</div> : <p>Login first</p>;
}

```

**Pattern:** Disable queries when conditions aren't met!

## Chapter 7

# Changing Data with Mutations

### 7.1 What is a Mutation?

Changing data on the server (create, update, delete).

```
// Read operation (Query)
GET /api/todos

// Write operation (Mutation)
POST /api/todos (create)
PUT /api/todos/1 (update)
DELETE /api/todos/1 (delete)
```

### 7.2 Using Mutations

```
import { useMutation } from '@tanstack/react-query';

function CreateTodo() {
  const mutation = useMutation({
    mutationFn: (newTodo) => {
      return fetch('/api/todos', {
        method: 'POST',
        body: JSON.stringify(newTodo)
      }).then(r => r.json());
    }
  });

  const handleSubmit = async (title) => {
    mutation.mutate({ title });
  };

  return (
    <div>
      <button onClick={() => handleSubmit('New Todo')}>
        Create
      </button>
    </div>
  );
}
```

```
    </button>
    {mutation.isPending && <p>Creating...</p>}
    {mutation.isError && <p>Error: {mutation.error.message}</p>}
    {mutation.isSuccess && <p>Created!</p>}
  </div>
);
}
```

## 7.3 Mutation States

| State     | Meaning             |
|-----------|---------------------|
| isPending | Request in progress |
| isError   | Request failed      |
| isSuccess | Request succeeded   |
| isIdle    | Not yet called      |

## 7.4 Mutation Data

Access mutation results:

```
const mutation = useMutation({
  mutationFn: createTodo
});

mutation.mutate({ title: 'New Todo' });

// After success:
console.log(mutation.data); // Server response
console.log(mutation.error); // Any error
```

## 7.5 Mutation with Variables

Pass data to mutation:

```
const deleteMutation = useMutation({
  mutationFn: (todoId) => {
    return fetch(`/api/todos/${todoId}`, {
      method: 'DELETE'
    });
  }
});

// Pass data when calling
deleteMutation.mutate(123); // Delete todo 123
```

**Pattern:** Use mutations for create/update/delete!

## Chapter 8

# Fetching More Data & Testing the Mutation

### 8.1 Fetching in Stages

Pagination and infinite queries:

```
import { useInfiniteQuery } from '@tanstack/react-query';

function InfiniteTodos() {
  const { data, fetchNextPage, hasNextPage } = useInfiniteQuery({
    queryKey: ['todos'],
    queryFn: ({ pageParam = 1 }) => fetchTodos(pageParam),
    getNextPageParam: (lastPage) => {
      return lastPage.hasMore ? lastPage.nextPage : undefined;
    }
  });

  return (
    <div>
      {data?.pages.map((page) =>
        page.todos.map(todo => <div key={todo.id}>{todo.title}</div>)
      )}
      {hasNextPage && (
        <button onClick={() => fetchNextPage()}>Load More</button>
      )}
    </div>
  );
}
```

### 8.2 Testing Mutations

Test your mutation works:

```
// Create test component
```

```
function CreateTodoTest() {
  const { mutate, isPending, data } = useMutation({
    mutationFn: createTodo
  });

  return (
    <div>
      <button onClick={() => mutate({ title: 'Test' })}>
        Create
      </button>
      {isPending && <p>Loading...</p>}
      {data && <p>Created: {data.id}</p>}
    </div>
  );
}

// Or write actual tests with testing-library
import { render, screen } from '@testing-library/react';

test('mutation creates todo', async () => {
  render(<CreateTodoTest />);
  const button = screen.getByText('Create');
  button.click();

  // Wait for success
  await screen.findByText(/Created:/);
});
```

## 8.3 Manual Fetch

Fetch without component:

```
import { useQueryClient } from '@tanstack/react-query';

function ManualFetch() {
  const queryClient = useQueryClient();

  const handleFetch = async () => {
    const data = await queryClient.fetchQuery({
      queryKey: ['todos'],
      queryFn: fetchTodos
    });
    console.log(data);
  };

  return <button onClick={handleFetch}>Fetch</button>;
}
```

**Testing:** Manual fetch useful for debugging!

## Chapter 9

# Acting on Mutation Success & Invalidating Queries

### 9.1 Callbacks on Success

Execute code after mutation succeeds:

```
const mutation = useMutation({
  mutationFn: createTodo,
  onSuccess: (data) => {
    console.log('Created todo:', data);
    // Show success message
    // Redirect user
    // Update UI
  },
  onError: (error) => {
    console.log('Error:', error);
    // Show error message
  }
});
```

### 9.2 Invalidating Queries

Tell React Query to refetch data:

```
import { useQueryClient } from '@tanstack/react-query';

const queryClient = useQueryClient();

const mutation = useMutation({
  mutationFn: createTodo,
  onSuccess: () => {
    // After creating, refetch todos list
    queryClient.invalidateQueries({
      queryKey: ['todos']
    })
  }
});
```

```
    });
  }
});
```

## 9.3 Complete Example

```
function CreateTodo() {
  const queryClient = useQueryClient();

  const { mutate, isPending } = useMutation({
    mutationFn: async (newTodo) => {
      const response = await fetch('/api/todos', {
        method: 'POST',
        body: JSON.stringify(newTodo)
      });
      return response.json();
    },
    onSuccess: (newTodo) => {
      // 1. Show success message
      console.log('Created:', newTodo);

      // 2. Update cache manually
      queryClient.setQueryData(['todos'], (old) => [
        ...old,
        newTodo
      ]);

      // 3. Or invalidate to refetch
      queryClient.invalidateQueries({ queryKey: ['todos'] });
    },
    onError: (error) => {
      console.error('Failed:', error);
    }
  });

  return (
    <button onClick={() => mutate({ title: 'New' })}>
      {isPending ? 'Creating...' : 'Create'}
    </button>
  );
}
```

## 9.4 Partial Invalidation

Invalidate only certain queries:

```
// Invalidate all 'todos' queries
```

```
queryClient.invalidateQueries({ queryKey: ['todos'] });

// Invalidate specific todo
queryClient.invalidateQueries({
  queryKey: ['todos', todoId]
});

// Invalidate with filter
queryClient.invalidateQueries({
  predicate: (query) => query.queryKey[0] === 'todos'
});
```

**Pattern:** After mutation success, invalidate related queries!

## Chapter 10

# Disabling Automatic Refetching After Invalidations

### 10.1 Default Behavior

When you invalidate a query, React Query automatically refetches:

```
// This triggers automatic refetch
queryClient.invalidateQueries({ queryKey: ['todos'] });

// Component re-renders with isLoading = true
// New data fetched from server
// Component re-renders with new data
```

### 10.2 Disabling Refetch

Prevent automatic refetch:

```
queryClient.invalidateQueries({
  queryKey: ['todos'],
  refetchType: 'none' // Don't refetch
});

// Query marked as stale but not refetched
// Next component render will refetch (lazy)
```

### 10.3 Refetch Types

| Type     | Behavior                                  |
|----------|-------------------------------------------|
| active   | Refetch only active queries (shown)       |
| inactive | Refetch only inactive queries (not shown) |
| all      | Refetch all queries (default)             |
| none     | Don't refetch                             |

## 10.4 Example: Optimistic Approach

```
const mutation = useMutation({
  mutationFn: updateTodo,
  onSuccess: () => {
    // Just mark as stale, don't refetch yet
    queryClient.invalidateQueries({
      queryKey: ['todos'],
      refetchType: 'none'
    });

    // Data will refetch when user navigates
    // or when component re-mounts
  }
});
```

## 10.5 When to Use

- Large lists (avoid big refetch)
- Slow API (avoid waiting)
- Already updated UI optimistically
- Immediate UX more important than freshness

**Trade-off:** Speed vs freshness!

## Chapter 11

# Enhancing the Demo App & Repeating Mutation Concepts

### 11.1 Complete Todo App Example

```
import { useQuery, useMutation, useQueryClient }
  from '@tanstack/react-query';

function TodoApp() {
  const queryClient = useQueryClient();

  // Fetch all todos
  const { data: todos = [], isLoading } = useQuery({
    queryKey: ['todos'],
    queryFn: () => fetch('/api/todos').then(r => r.json())
  });

  // Create todo mutation
  const createMutation = useMutation({
    mutationFn: (title) =>
      fetch('/api/todos', {
        method: 'POST',
        body: JSON.stringify({ title })
      }).then(r => r.json()),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['todos'] });
    }
  });

  // Delete todo mutation
  const deleteMutation = useMutation({
    mutationFn: (todoId) =>
      fetch(`/api/todos/${todoId}`, { method: 'DELETE' }),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['todos'] });
    }
  });
}
```

```
    }
  });

  // Update todo mutation
  const updateMutation = useMutation({
    mutationFn: ({ id, ...data }) =>
      fetch(`/api/todos/${id}`, {
        method: 'PUT',
        body: JSON.stringify(data)
      }).then(r => r.json()),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['todos'] });
    }
  });

  if (isLoading) return <p>Loading...</p>;

  return (
    <div>
      <h1>Todos</h1>

      <div>
        <input placeholder="New todo" />
        <button onClick={e => {
          const title = e.target.previousSibling.value;
          createMutation.mutate(title);
        }}>
          Add
        </button>
      </div>

      <ul>
        {todos.map(todo => (
          <li key={todo.id}>
            <span>{todo.title}</span>
            <button onClick={() => deleteMutation.mutate(todo.id)}>
              Delete
            </button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

## 11.2 Key Mutation Concepts

1. Define mutation function
2. Call with `mutate()`
3. Handle success/error
4. Invalidate related queries
5. UI updates automatically

**Complete:** Full CRUD with React Query!

## Chapter 12

# React Query Advantages In Action

### 12.1 Before: Manual Fetching

```
const [todos, setTodos] = useState([]);
const [loading, setLoading] = useState(false);
const [error, setError] = useState(null);

useEffect(() => {
  setLoading(true);
  fetch('/api/todos')
    .then(r => r.json())
    .then(data => {
      setTodos(data);
      setLoading(false);
    })
    .catch(err => {
      setError(err);
      setLoading(false);
    });
}, []);

// Plus manage: caching, refetching, updates...
// Lots of code!
```

### 12.2 After: React Query

```
const { data: todos = [], isLoading, error } = useQuery({
  queryKey: ['todos'],
  queryFn: () => fetch('/api/todos').then(r => r.json())
});

// That's it! Caching, refetching, updates all included!
```

## 12.3 Advantage 1: No State Management

- No useState for data
- No useState for loading
- No useState for error
- React Query handles it

## 12.4 Advantage 2: Automatic Caching

- Visit page → fetch
- Leave page → cached
- Return to page → instant (if not stale)
- No loading spinner

## 12.5 Advantage 3: Automatic Refetching

- User tabs away and back → refetch
- User reconnects to internet → refetch
- Configurable intervals
- Always fresh data

## 12.6 Advantage 4: Easy Mutations

- Create, update, delete easily
- Automatic retry on failure
- Automatic UI refresh
- Optimistic updates possible

## 12.7 Advantage 5: Better DevTools

React Query DevTools show:

- All queries and their state
- When queries refresh
- Cache contents
- Time travel debugging

**Result:** Simpler code, better UX, less bugs!

## Chapter 13

# Updating Data with Mutations

### 13.1 Update Pattern

Update existing data:

```
const updateMutation = useMutation({
  mutationFn: ({ id, title }) => {
    return fetch(`/api/todos/${id}`, {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ title })
    }).then(r => r.json());
  },
  onSuccess: (updatedTodo) => {
    // Update cache
    queryClient.setQueryData(['todos'], (old) =>
      old.map(todo =>
        todo.id === updatedTodo.id ? updatedTodo : todo
      )
    );
  }
});

// Use
updateMutation.mutate({ id: 1, title: 'Updated Title' });
```

### 13.2 Delete Pattern

Remove data:

```
const deleteMutation = useMutation({
  mutationFn: (todoId) => {
    return fetch(`/api/todos/${todoId}`, {
      method: 'DELETE'
    });
  },
});
```

```
onSuccess: () => {
  // Invalidate to refetch
  queryClient.invalidateQueries({ queryKey: ['todos'] });
}
});

// Use
deleteMutation.mutate(todoId);
```

### 13.3 Bulk Update

Update multiple items:

```
const bulkUpdateMutation = useMutation({
  mutationFn: (todoIds) => {
    return fetch('/api/todos/bulk-update', {
      method: 'POST',
      body: JSON.stringify({ ids: todoIds, completed: true })
    });
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['todos'] });
  }
});
```

**Patterns:** All CRUD operations covered!

## Chapter 14

# Optimistic Updating & Query Keys with React Router

### 14.1 Optimistic Updating

Update UI before server confirms:

```
const updateMutation = useMutation({
  mutationFn: updateTodo,
  onMutate: async (newTodo) => {
    // Cancel previous fetches
    await queryClient.cancelQueries({ queryKey: ['todos'] });

    // Snapshot old data
    const oldData = queryClient.getQueryData(['todos']);

    // Optimistically update
    queryClient.setQueryData(['todos'], (old) =>
      old.map(todo =>
        todo.id === newTodo.id ? newTodo : todo
      )
    );

    return { oldData }; // Save for rollback
  },
  onError: (error, newTodo, context) => {
    // Rollback on error
    queryClient.setQueryData(['todos'], context.oldData);
  },
  onSuccess: () => {
    // Confirm success (optional)
    queryClient.invalidateQueries({ queryKey: ['todos'] });
  }
});

// User sees update instantly
```

```
// If server fails, rolls back automatically
```

## 14.2 Using Query Keys with React Router

Dynamic query keys from route params:

```
import { useParams } from 'react-router-dom';

function TodoDetail() {
  const { todoId } = useParams();

  const { data } = useQuery({
    queryKey: ['todos', todoId], // Include route param
    queryFn: () => fetchTodo(todoId),
    enabled: !!todoId
  });

  return <div>{data?.title}</div>;
}

// Each URL has own cache
// /todos/1 cached separately from /todos/2
```

## 14.3 Route Change Refetch

Refetch when route changes:

```
function RouteAwareComponent() {
  const { userId } = useParams();

  const { data } = useQuery({
    queryKey: ['user', userId],
    queryFn: () => fetchUser(userId)
  });

  // React Query automatically refetches
  // when userId changes (new route)

  return <div>{data?.name}</div>;
}

// User goes from /user/1 to /user/2
// React Query automatically fetches new user
```

## 14.4 Complete Example

```
function EditTodo() {
```

```

const { todoId } = useParams();
const navigate = useNavigate();
const queryClient = useQueryClient();

const { data: todo } = useQuery({
  queryKey: ['todos', todoId],
  queryFn: () => fetchTodo(todoId)
});

const mutation = useMutation({
  mutationFn: (updated) => updateTodo(todoId, updated),
  onSuccess: () => {
    // Invalidate this specific todo
    queryClient.invalidateQueries({
      queryKey: ['todos', todoId]
    });
    // Invalidate todos list
    queryClient.invalidateQueries({
      queryKey: ['todos']
    });
    navigate('/todos');
  }
});

return (
  <form onSubmit={(e) => {
    e.preventDefault();
    const title = e.target.title.value;
    mutation.mutate({ title });
  }}>
    <input name="title" defaultValue={todo?.title} />
    <button>Save</button>
  </form>
);
}

```

**Integration:** React Query + React Router = Perfect!

# Quick Summary

## React Query Essentials

1. Install: `npm install @tanstack/react-query`
2. Wrap app with `QueryClientProvider`
3. Use `useQuery` for fetching
4. Use `useMutation` for updates
5. Use `useQueryClient` for cache management

## Core Concepts

- **Query** - Get data (read)
- **Mutation** - Change data (write)
- **Cache** - Store fetched data
- **Stale** - Data is old
- **Invalid** - Refetch needed

## Configuration

- `staleTime` - How long data is fresh
- `gcTime` - How long to keep in memory
- `refetchOnMount` - Refetch when mounted
- `refetchOnWindowFocus` - Refetch on tab focus
- `retry` - How many times to retry

## Best Practices

1. Use descriptive query keys
2. Invalidate queries after mutations
3. Handle loading and error states
4. Use optimistic updates for better UX
5. Combine with React Router naturally
6. Use QueryClientProvider wrapper
7. Test with React Query DevTools

## Advantages Over Manual Fetching

- Less code to write
- Automatic caching
- Automatic refetching
- Better error handling
- Better loading states
- Easier mutations
- DevTools for debugging

## Common Patterns

| Pattern            | Use Case         |
|--------------------|------------------|
| Query + Mutation   | CRUD operations  |
| Dependent Queries  | Chain requests   |
| Infinite Queries   | Pagination       |
| Optimistic Updates | Fast UX          |
| Query Keys         | Cache management |

## Debugging

- Install React Query DevTools
- See all queries and state
- Time travel through actions
- Inspect cache contents
- Test refetching behavior

---

*React Query = Less code, better performance, happier developers!*